

WEEK-6

TASK 1: Create a client-server application using Named Pipes (FIFOs). The client sends messages and the Server processes and responds to them. Analyze FIFO behavior when multiple clients communicate simultaneously

```
ramya@LAPTOP-OPDAF309: ~$ nano server.c
GNU nano 8.7.1 server.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/stat.h>

#define FIFO1 "client_to_server"
#define FIFO2 "server_to_client"

int main()
{
    char message[100];

    mkfifo(FIFO1, 0666);
    mkfifo(FIFO2, 0666);

    printf("Server started. Waiting for client...\n");

    int fd_read = open(FIFO1, O_RDONLY);
    int fd_write = open(FIFO2, O_WRONLY);

    while (1)
    {
        memset(message, 0, sizeof(message));

        read(fd_read, message, sizeof(message));

        if (strcmp(message, "exit") == 0)
        {
            printf("Client disconnected.\n");
            break;
        }

        printf("Client: %s\n", message);
    }
}
```

```
char response[150];
snprintf(response, sizeof(response),
          "Server processed: %s", message);

write(fd_write, response, strlen(response) + 1);
}

close(fd_read);
close(fd_write);

unlink(FIFO1);
unlink(FIFO2);

return 0;
```

```
GNU nano 8.7.1 client.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>

#define FIF01 "client_to_server"
#define FIF02 "server_to_client"

int main()
{
    char message[100];
    char response[150];

    int fd_write = open(FIF01, O_WRONLY);
    int fd_read = open(FIF02, O_RDONLY);

    while (1)
    {
        printf("Enter message: ");
        fgets(message, sizeof(message), stdin);

        message[strcspn(message, "\n")] = '\0';

        write(fd_write, message, strlen(message) + 1);

        if (strcmp(message, "exit") == 0)
            break;

        memset(response, 0, sizeof(response));
        read(fd_read, response, sizeof(response));

        printf("Server: %s\n", response);
    }
}
```

```
close(fd_write);  
close(fd_read);  
  
return 0;  
}
```

```
ramya@LAPTOP-OPDAF309: ~$ mkdir Prac4  
ramya@LAPTOP-OPDAF309: ~$  
ramya@LAPTOP-OPDAF309: ~$ cd Prac4  
ramya@LAPTOP-OPDAF309: ~/Prac4$ nano server.c  
ramya@LAPTOP-OPDAF309: ~/Prac4$ gcc server.c -o server  
ramya@LAPTOP-OPDAF309: ~/Prac4$ nano client.c  
ramya@LAPTOP-OPDAF309: ~/Prac4$ gcc client.c -o client  
ramya@LAPTOP-OPDAF309: ~/Prac4$ ./server &  
[1] 613  
Server started. Waiting for client...  
ramya@LAPTOP-OPDAF309: ~/Prac4$ ./client  
Enter message: Hello Server  
Client: Hello Server  
Server: Server processed: Hello Server  
Enter message: How are you?  
Client: How are you?  
Server: Server processed: How are you?  
Enter message: exit  
Client disconnected.  
[1]+ Done  
ramya@LAPTOP-OPDAF309: ~/Prac4$ |
```

```
ramya@LAPTOP-OPDAF309: ~$ nano multi_client.c
GNU nano 8.7.1 multi_client.c *
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <string.h>

#define FIFO1 "client_to_server"
#define FIFO2 "server_to_client"

int main(int argc, char *argv[])
{
    char message[100];
    char response[150];

    int fd_write = open(FIFO1, O_WRONLY);
    int fd_read = open(FIFO2, O_RDONLY);

    snprintf(message, sizeof(message),
              "Message from Client %s", argv[1]);

    write(fd_write, message, strlen(message) + 1);

    read(fd_read, response, sizeof(response));

    printf("Client %s received: %s\n", argv[1], response);

    close(fd_write);
    close(fd_read);

    return 0;
}
```

```
ramya@LAPTOP-OPDAF309:~/Prac4$ ./multi_client 1 &
./multi_client 2 &
[3] 763
Client: Message from Client 1
Client 1 received: Server processed: Message from Client 1
Client:
Client:
[4] 769
[1] Broken pipe ./server
[2] Broken pipe ./server
[3] Done ./multi_client 1
```

TASK 2: Create a POSIX signal handling program that captures SIGINT, SIGTERM, and SIGUSR1. Demonstrate asynchronous event handling and explain the role of signal handlers. please give the commands for this and i need everything in one terminal only

```
ramya@LAPTOP-OPDAF309: ~$ nano signals.c
GNU nano 8.7.1 signals.c *
#include <stdio.h>
#include <stdlib.h>
#include <signal.h>
#include <unistd.h>

void handle_signal(int sig)
{
    if (sig == SIGINT)
        printf("\nReceived SIGINT (Ctrl+C)\n");

    else if (sig == SIGTERM)
        printf("\nReceived SIGTERM\n");

    else if (sig == SIGUSR1)
        printf("\nReceived SIGUSR1\n");

    printf("Signal handler executed asynchronously.\n");
}

int main()
{
    signal(SIGINT, handle_signal);
    signal(SIGTERM, handle_signal);
    signal(SIGUSR1, handle_signal);

    printf("Signal handling program started.\n");
    printf("Process ID (PID): %d\n", getpid());
    printf("Waiting for signals...\n");

    while (1)
    {
        printf("Program is running...\n");
        sleep(5);
    }

    return 0;
}
```

[illegible]